Database Development and Design (SOE201)

Lecture 3b: Advanced Indexing

Dr. Wei Wang
AIAT

Learning Outcomes

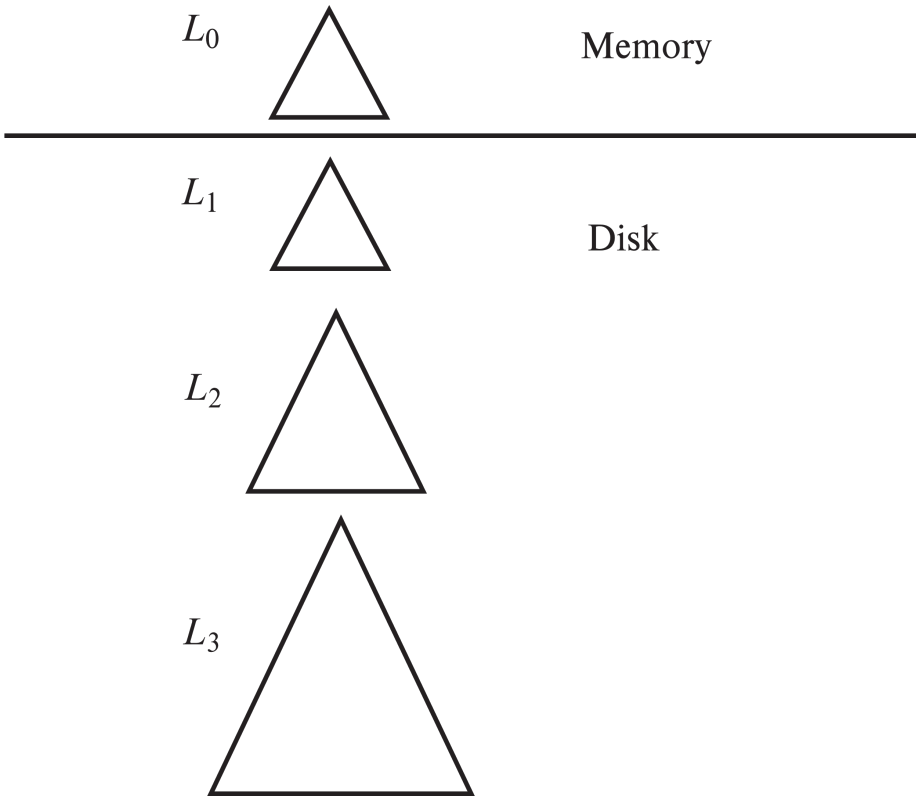
- Write optimised index
  - LSM tree
  - Buffer tree
- Bitmap index
- Bloom Filter
- Spatial Index
  - K-d tree
  - Quadtree
  - R-Tree
- Temporal Index

Write Optimised Index

- Performance of B^+-trees can be poor for write-intensive workload, e.g.
  - One I/O per leaf, assuming all internal nodes are in memory
  - With magnetic disks, < 100 inserts per second per disk
  - With flash memory, one page overwrite per insert
- Two approaches to reducing cost of writes
  - Log-structured merge (LSM) tree
  - Buffer tree

LSM Tree

- Consider only inserts/queries for now
- Records inserted first into in-memory tree (L_0 tree)
- When in-memory tree is full, records moved to disk (L_1 tree)
  - B^+-tree constructed using bottom-up built by merging existing L_1 tree with records from L_0 tree
- When L_1 tree exceeds some threshold, merge into L_2 tree
  - And so on for more levels
  - Size threshold for L_{i+1} tree is k times size threshold for L_i tree



LSM Tree – cont'd

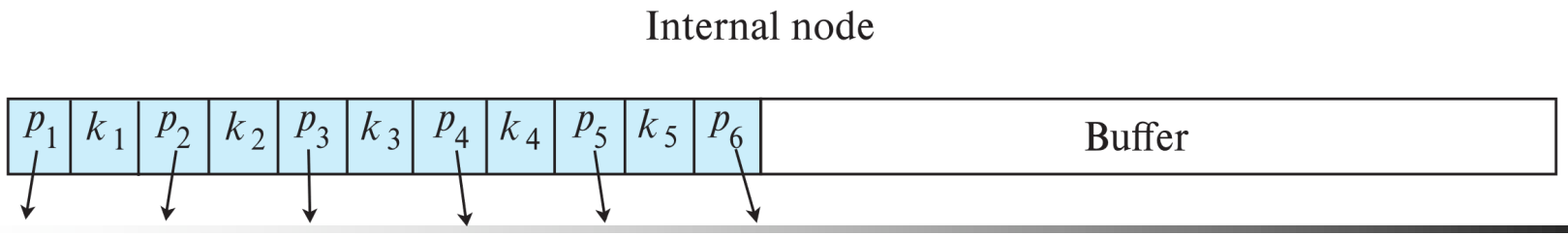
- Benefits of LSM
  - Inserts are done using only sequential I/O operations
  - Leaves are full, avoiding space wastage
  - Reduced number of I/O operations per record inserted as compared to normal B$^+$-tree (up to some sizes)
- Drawback of LSM
  - Queries have to search multiple trees
  - Entire content of each level copied multiple times
- Stepped-merge index
  - Variant of LSM tree with multiple trees at each level
  - Reduces write cost compared to LSM tree
  - But queries are even more expensive
    - Bloom filters to avoid lookups in most trees

LSM Tree – cont'd

- Deletion handled by adding special "delete" entries
  - Lookups will find both original entry and delete entry, and return only those entries that do not have matching delete entry.
  - When trees are merged, if we find delete entry matching an original entry, both are dropped.
- Update handled using insert+delete
- LSM trees were introduced for disk-based indices
  - But useful to minimise erases with flash-based indices
  - Stepped-merge variant of LSM trees is used in many BigData storage systems.
    - Google BigTable, Apache Cassandra, MongoDB
    - And more recently in SQLite4, LevelDB, and MyRocks storage engine of MySQL.

Buffer Tree

- Alternative to LSM tree
- Key idea: each internal node of B+-tree has buffer to store inserts
  - Inserts are first inserted into buffer of root
  - Inserts are moved to lower levels when buffer is full
  - With large buffer, many records are moved to lower level each time
- Benefits
  - Less overhead on queries; per record I/O decreases correspondingly
  - Can be used with any tree index structure
    - PostgreSQL Generalised Search Tree (GiST) indices
- Drawback: more random I/O than LSM tree



Bitmap Index

- Bitmap is simply array of bits
- Bitmap index is special type of index designed for efficient query on multiple keys
- Records in relation are assumed to be numbered sequentially
- Applicable on attributes that take on relatively small number of distinct values
  - e.g. gender, country, state, …
  - e.g. income-level (income broken up into small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000-infinity)

Bitmap Index cont'd

- In its simplest form, bitmap index on attribute has bitmap for each value of attribute
  - Bitmap has as many bits as records
  - In bitmap for value v, bit for record is 1 if record has value v for that attribute, and is 0 otherwise

record number	*ID*	*gender*	*income_level*
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1
3	15151	m	L4
4	58583	f	L3

Bitmaps for *gender*

m	10010
f	01101

Bitmaps for *income_level*

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

Bitmap Index cont'd

- Bitmap indices are useful for queries on multiple attributes
  - Not particularly useful for single attribute queries
  - Bitmap indices generally very small compared with relation size
- Queries are answered using bitmap operations:
  Intersection (and); Union (or); and Negation (not)
- Example: query takes two bitmaps and applies operation on them to get result, e.g.
  - For query "males with income level L1": 10010 AND 10100 = 10000
    - Can then retrieve required tuples.
    - Counting number of matching tuples is even faster

Bloom Filter

- Bloom filter is probabilistic data structure used to check membership of value in set
  - May return true (with low probability) even if element is not present
  - But never returns false if element is present
  - Used to filter out irrelevant sets
- Key data structure is single bitmap
  - For set with *n* elements, typical bitmap size is *10n*
- With single hash function *h*() with range=number of bits in bitmap:
  - For each element *s* in set *S* compute *h*(*s*) and set bit *h*(*s*)
  - To query element *v* compute *h*(*v*), and check if bit *h*(*v*) is set

Bloom Filter – cont'd

- Problem with single hash function: significant chance of false positive due to hash collision
  - 10% chance with $10n$ bits
- Key idea of Bloom filter: reduce false positives by using multiple independent hash functions $h_i()$ for $i = 1..k$
  - For each element s in set S; *for each* i compute $h_i(s)$ and set bit $h_i(s)$
  - To query an element v *for each* i compute $h_i(v)$, and check if bit $h_i(v)$ is set
    - If bit $h_i(v)$ is set for every i then report v as present in set
    - Else report v as absent
  - With $10n$ bits, and $k = 7$, false positive rate reduces to 1%

Spatial Index

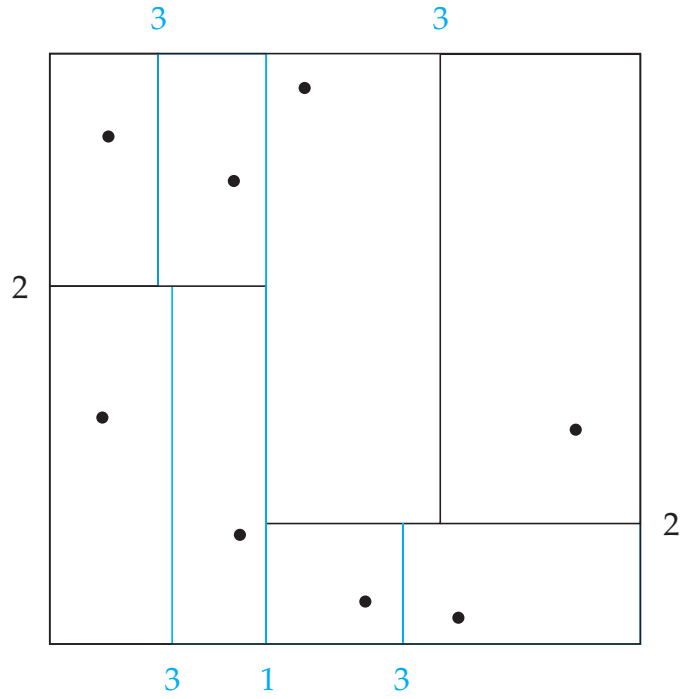
- Databases can also store data types such as lines and polygons, in addition to raster images
  - Allows relational databases to store and retrieve spatial information
  - Queries can use spatial conditions (e.g. contains or overlaps).
  - Queries can mix spatial and non-spatial conditions

Spatial Index – cont'd

- Nearest neighbor queries - given point or object, find nearest objects that satisfy given conditions.
- Range queries - deal with spatial regions, e.g. ask for objects that lie partially or fully inside specified region.
  - e.g. queries that compute intersections or unions of regions.
- Spatial join of two spatial relations, with location playing role of join attribute.

K-d Tree

- Each level of *k-d* tree partitions space into two.
  - Choose one dimension (i.e. x axis) for partitioning at root level of tree.
  - Choose another dimension for partitioning nodes at next level and so on, cycling through dimensions.
- In each node, approximately half of points stored in sub-tree fall on one side and half on other.
- Partitioning stops when node has less than given number of points.



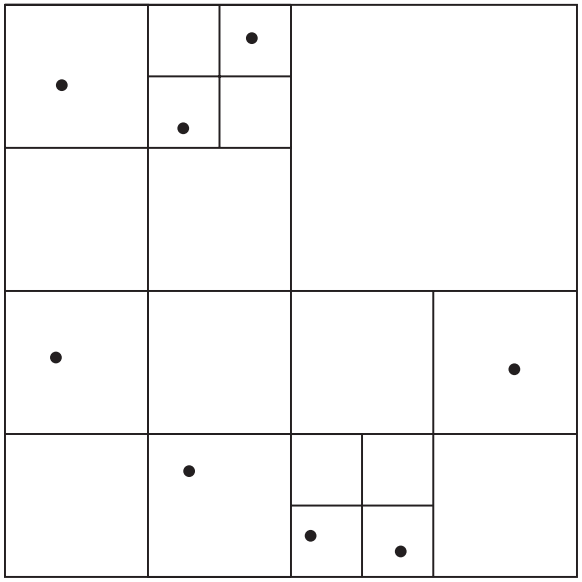
k-d-B tree extends *k-d* tree to allow multiple child nodes for each internal node.

K-d Tree – cont'd

- Range search can be performed by following recursive procedure, starting at root:
  - Suppose node is internal, and let it be split on particular dimension, say x, at point x_i. Entries in left subtree have x values < x_i, and those in right subtree have x values ≥ x_i.
    - If query range contains x_i, search is recursively performed on both children.
    - If query range is to left of x_i, search is recursively performed only on left child.
    - Otherwise it is performed only on right subtree.
  - If node is leaf, all entries that are contained in query range are retrieved.

Quadtree

Each node of quadtree is associated with rectangular region of space; top node is associated with entire target space.



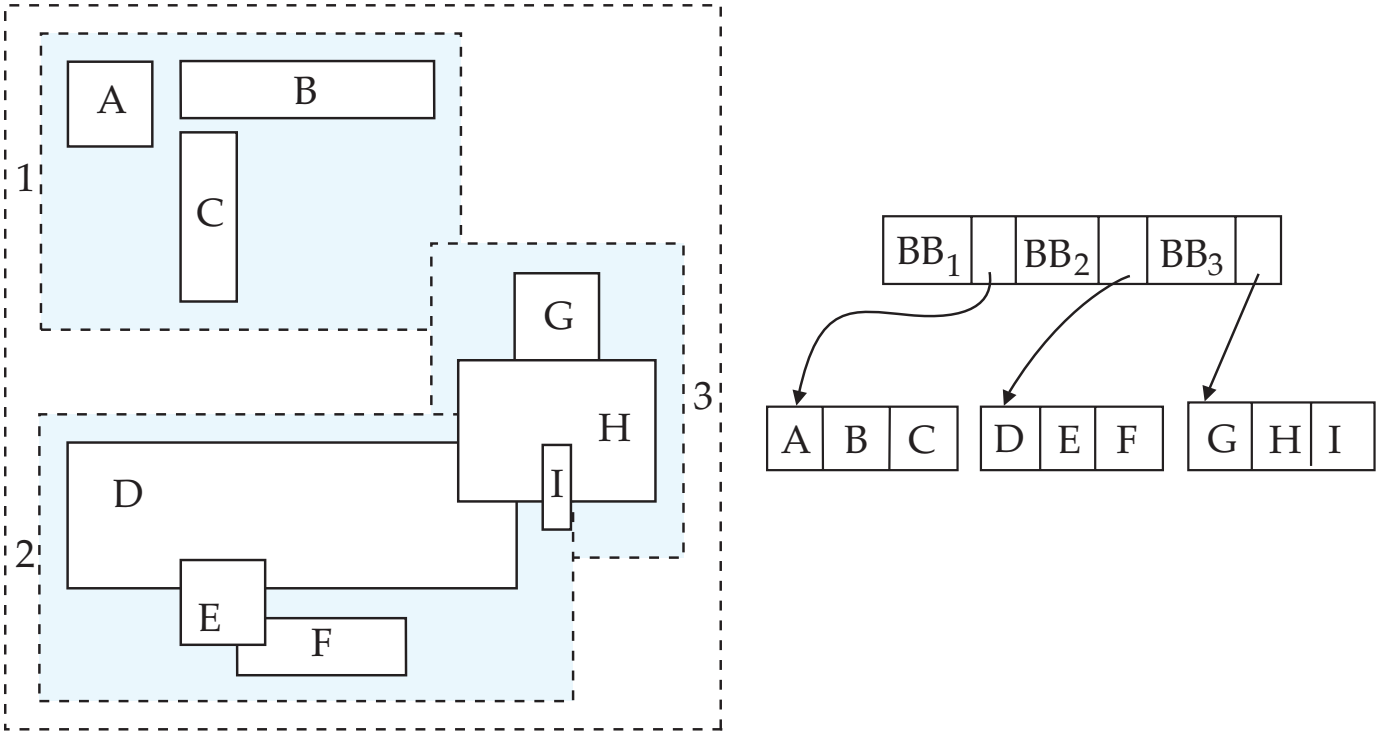
- Each non-leaf nodes divides its region into four equal sized quadrants
  - Each such node has four child nodes corresponding to four quadrants and so on
- Leaf nodes have between zero and some fixed maximum number of points (set to 1 in example).

R-Tree

- R-tree is useful for indexing sets of rectangles and other polygons.
- Supported in many modern database systems, along with variants like R^+-tree and R*-tree.
- Basic idea: instead of range of values, rectangular bounding box is associated with each tree node.
- Bounding box is minimum-sized rectangle that contains all rectangles/polygons associated with node
  - Bounding boxes of children of node are allowed to overlap, e.g. two-dimensional case ($N = 2$)
  - Generalisation for $N > 2$ is straightforward, although R-trees work well only for relatively small N

R-Tree – cont'd

- Left: map including set of rectangles (solid line) and bounding boxes (dashed line) of nodes.
- Right: R-tree



R-Tree – cont'd

- To find data items intersecting given query point/region, starting from root:
  - If node is leaf, output data items whose keys intersect given query point/region.
  - Else, for each child of current node whose bounding box intersects query point/region, recursively search child
- Can be very inefficient in worst case since multiple paths may need to be searched, but works acceptably in practice.

Temporal Index

- Temporal data refers to data that has associated time period (interval)
  - Example: temporal version of *course* relation

course_id	*title*	*dept_name*	*credits*	*start*	*end*
BIO-101	Intro. to Biology	Biology	4	1985-01-01	9999-12-31
CS-201	Intro. to C	Comp. Sci.	4	1985-01-01	1999-01-01
CS-201	Intro. to Java	Comp. Sci.	4	1999-01-01	2010-01-01
CS-201	Intro. to Python	Comp. Sci.	4	2010-01-01	9999-12-31

- Time interval has start and end time
  - End time set to infinity (or large date such as 9999-12-31) if tuple is currently valid and its validity end time is not currently known
- Query may ask for all tuples that are valid at point in time or during time interval
  - Index on valid time period speeds up this task

Temporal Index – cont'd

- To create temporal index on attribute *a*:
  - Use spatial index, such as R-tree, with attribute *a* as one dimension, and time as another dimension
  - Problems: end time is infinite or very large.
  - Solution:
    - Store all current tuples (with end time as infinity) in separate index, indexed on (*a, start-time*), using B^+-Tree for example.
    - Non-current tuples would be spatial index such as R-tree.
    - Lookups need to search on both indices.
    - To find tuples valid at point in time t_i in current tuple index, search for tuples with $a = v$, and $t_s \leq t_i$,

End of Lecture

- Reading
  - Textbook, chapter 14 and 24